

Handling Grammar Cycles in the 1997 SML Definition

Elizabeth Scott

Computer Science

Royal Holloway, University of London

Egham, Surrey, UK

e.scott@rhul.ac.uk

Adrian Johnstone

Computer Science

Royal Holloway, University of London

Egham, Surrey, UK

a.johnstone@rhul.ac.uk

Abstract

Fully general parsers permit the syntax specification of formal languages to be unrestricted, allowing language designers to use a syntax specification that supports semantics specification, but also permitting ambiguity. Language workbenches that support fully general grammars need tools which can generate robust default behaviour but also allow experienced designers to specify particular choices when ambiguity is encountered. The standard longest match approach to ambiguity resolution is not robust when a grammar contains cycles. Although cycles can be removed from a grammar, this disrupts the syntax specification. We present an algorithm that safely removes cycles from the shared packed parse forests generated by general parsers and explore the application of the algorithm to the 1997 SML Definition. The algorithm results in a sub-forest to which further designer-specified or default disambiguation rules can be applied.

CCS Concepts: • Software and its engineering → Formal language definitions.

Keywords: grammar ambiguity, generalised parsing, syntax analysis, derivation trees, standard ML

ACM Reference Format:

Elizabeth Scott and Adrian Johnstone. 2025. Handling Grammar Cycles in the 1997 SML Definition. In *Proceedings of the 18th ACM SIGPLAN International Conference on Software Language Engineering (SLE '25)*, June 12–13, 2025, Koblenz, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3732771.3742709>

1 Introduction

The emergence of efficient fully general parsers, for example the generalised LR based parsers [16] generated by ASF+SDF [17] and Spoofax [7] and the GLL style generalisations of recursive descent parsers [11, 12, 15] generated by ART [5] and RASCAL [8], permit the syntax specification of formal languages to be unrestricted. This allows language

designers to use a syntax specification that supports their desired semantics specification, but the freedom comes at a price. For example, permitting syntactic ambiguity creates a requirement for a later stage to resolve the corresponding semantic ambiguity. Whereas, if the parsing technology requires a grammar to be non-ambiguous, or at least for ambiguity to be limited to situations with well established handling methods, the semantics specification will not have to provide a resolution.

Language workbenches that support fully general grammars should ideally provide users with tools that can generate robust default behaviour but also allow experienced designers to specify particular choices when ambiguity is encountered. Standard approaches to ambiguity resolution, such as longest match, are not robust when a grammar contains cycles. Although cycles can be removed from a grammar, this disrupts the syntax specification. Grammar ambiguity is undecidable in general but, for a given input string, ambiguity can be identified by the existence of multiple packed node children in the output shared packed parse forest [3] (SPPF) representation of the derivation trees. While ambiguity resolution can thus be seen as the removal of SPPF packed nodes, this is problematic in the case of cycles as removing nodes may leave a graph which does not embed any derivations of the input.

To provide insight into the issues that can arise when fully general language specifications are deployed we can consider the 1997 Definition of Standard ML [9] which was designed to support compiler generation. This Definition has already been used to illustrate the way that lexical ambiguity can be addressed using multi-lexer parsing [13]. For that work a hand written cycle breaking specification was included in the post-parse analysis in order to produce experimental data comparing the multi-lexer and character level parsing approaches. That cycle removal specification is in terms of particular SML grammar rules and does not directly extend even to alternative SML grammars. In this paper we present a fully general algorithm that safely removes cycles from the SPPFs generated by general parsers, and we investigate its application to the SML Definition. The algorithm results in a graph to which further designer-specified or default disambiguation rules can be applied as desired.

In this paper we (i) give an algorithm that removes all cycles from the SPPF for any string and grammar, without removing all its derivations, and (ii) investigate cycles and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SLE '25, Koblenz, Germany

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1884-7/25/06

<https://doi.org/10.1145/3732771.3742709>

cycle removal for SPPFs generated from the 1997 Definition grammar for SML.

We begin with some background terminology and, as motivation, a brief discussion of the 1997 SML Definition. In Section 3 we describe binarised SPPFs and give examples that illustrate the issues that cycle removal can trigger. In Section 4 we describe an algorithm that traverses an SPPF and identifies nodes that can be removed to obtain a cycle-free graph that still embeds only derivations of the input. In Section 5 we discuss the cycles that arise in the SML Definition grammars.

2 Background

Where applicable, we follow the terminology and definitions in Aho *et. al.* [2]. A *context free grammar*, Γ , consists of a set, $\mathbf{T}$, of terminals, a set, $\mathbf{N}$, of nonterminals disjoint from $\mathbf{T}$, a start symbol, $S \in \mathbf{N}$, and a set of grammar rules $X ::= \alpha_1 \mid \dots \mid \alpha_t$, one for each nonterminal $X \in \mathbf{N}$, where $\alpha_k \in (\mathbf{T} \cup \mathbf{N})^*$. We refer to the α_k as the *production alternates*, or just *alternates*, of X , and to $X ::= \alpha_k$ as a *production rule*. We allow α_k to be the empty string, which is denoted by ϵ . We refer to a position at the start, end or between two symbols in an alternate as a *grammar slot* and we denote slots using a ‘dot’ in the same way as an LR item. So $X ::= x_1 \dots x_{i-1} \cdot x_i \dots x_p$ denotes a slot in the production rule $X ::= x_1 \dots x_p$.

A *derivation step* is an expansion $\gamma Y \beta \Rightarrow \gamma \alpha \beta$ where $\gamma, \beta \in (\mathbf{T} \cup \mathbf{N})^*$ and α is an alternate of Y . A *derivation* of τ from σ is a sequence $\sigma \Rightarrow \beta_1 \Rightarrow \dots \Rightarrow \beta_{n-1} \Rightarrow \tau$, also written $\sigma \xRightarrow{*} \tau$. We may write $\sigma \xRightarrow{+} \tau$ to indicate that the derivation contains at least one step.

Some of our proofs will be by contradiction on derivation distance. If $\alpha \xRightarrow{*} \beta$ then we call the smallest integer m such that $\alpha \xRightarrow{m} \beta$ the *derivation distance* of β from α . We shall extend the definition of derivation distance to graph nodes when we define SPPFs below.

A derivation is *left-most* if the left most nonterminal is replaced at each step and a nonterminal, X , is *ambiguous* if there is more than one left-most derivation of some string τ from X .

A nonterminal X is *cyclic* if $X \xRightarrow{+} X$ and a slot $X ::= \alpha \cdot \beta$ is cyclic if $\alpha \xRightarrow{*} X$ and $\beta \xRightarrow{*} \epsilon$. Thus cyclic slots are of the form $X ::= \gamma_1 Y \gamma_2 \cdot \gamma_3$ where $\gamma_i \xRightarrow{*} \epsilon$ and $Y \xRightarrow{*} X$.

The language, $L(x)$, generated by grammar symbol x is the set of $t \in \mathbf{T}^*$ such that $x \xRightarrow{*} t$. The language, $L(\Gamma)$, defined by the grammar Γ is the language generated by the start symbol of Γ .

2.1 The 1997 SML Definition

In the Definition of Standard ML [9], the syntax and semantics have been formally specified in a way that is designed to support the generation of a compiler. The phrase level syntax is specified using a context free grammar, annotated

with some comments, whose terminals are lexical tokens. We refer to this grammar as `fullSMLphrase.bnf`. The 1997 Definition also includes another grammar, `bareSMLphrase.bnf`, specifying a smaller language which is essentially a subset of the full SML language. There is a map from the full SML language to the bare SML language defined as a set of rewrite rules which are expressed in terms of the structure given by `fullSMLphrase.bnf`. Semantics are defined for bare SML in natural semantics style using SOS inference rules written in terms of the `bareSMLphrase.bnf` grammar rules. Thus the formal SML semantics are intricately dependent on the grammar used for the syntax specification.

Because the specification is not in a form that allows a parser to be directly implemented using classical deterministic approaches, actual implementations such as HaMLet [1] have required significant modification to the structure in the Definition. Modifications which enable such implementations have been studied extensively [6, 10] and our goal is not to replicate that work. Because the rewrite rules and the SOS rules are defined in terms of the grammar rules, we would like to be able to use the 1997 Definition *as written* as the specification from which a tool that executes semantic evaluation of SML can be directly generated.

To that end we are revisiting the SML Definition to consider to what extent modern tooling allows the syntax specification to be implemented exactly as originally written. The grammars we use for our study are the same as the 1997 SML Definition [9] except that we have converted EBNF optional constructs to BNF and sequences with ellipses, e.g. $(exp_1, \dots, exp_n)$, to recursively defined constructs. In this paper we focus on cycle removal, which is a required basis for the safe application of disambiguation strategies such as longest match.

Unlikely as it may seem in a grammar that specifies the syntax of a general programming language, the 1997 Definition of SML contains several cyclic production rules. This arises, for example, because declarations are allowed to be empty, and the syntax permits sequences of declarations to be both semicolon separated and for the semicolons to be omitted. In detail, SML program fragments are structured into ‘phrase classes’ that reflect the intended semantics. The grammar has a nonterminal for each phrase class and grammar rules that specify the sentences of that phrase class. A portion of `fullSMLphrase.bnf` is given in Figure 1 below.

For the purposes of this paper it is not necessary to understand either SML or the full details of the grammars in the 1997 Definition. However, we give a limited overview here of aspects of the grammar that impact on the cycle discussion in Section 5.

The phrase class *Program* is the set of full SML sentences, and *program* is the start nonterminal for both the full and bare grammars. An SML program is a semicolon delimited sequence of top level declarations, the phrase class *TopDec*, which is generated by the nonterminal *topdec*. The elements

```

program ::= topdec ; | topdec ; program
topdec ::= strdec | sigdec | fundec | strdec topdec
         | sigdec topdec | fundec topdec
strdec ::= structure strbind | dec | ε
         | strdec strdec | strdec ; strdec
fundec ::= functor funbind
sigdec ::= signature sigbind
spec ::= val valdesc | ε | spec spec | spec ; spec
valdesc ::= vid ; ty and tyndesc | vid ; ty
dec ::= val tyvarseq valbind | ε | dec dec | dec ; dec
valbind ::= pat = exp
tyvarseq ::= tyvar | ε | ( tyList )
tyList ::= tyvar | tyList , tyvar
exp ::= infexp | if exp then exp else exp
      | case exp of match
infexp ::= appexp | infexp vid infexp
appexp ::= atexp | appexp atexp
atexp ::= scon | longvid | { } | ( ) | ( exp )
longvid ::= vid | strid . longvid

```

Figure 1. Part of the SML grammar

of TopDec are grouped into subclasses StrDec, SigDec and FunDec (structure level, signature and functor declarations) with corresponding nonterminals *strdec*, *sigdec* and *fundec*. Of these only *strdec* can generate strings that do not contain at least one terminal, a necessary condition for the nonterminal to be nullable or cyclic. For *strdec* all the alternates that do not contain terminals are shown in the fragment in Figure 1 and we can focus on the phrase class Dec, which has corresponding nonterminal *dec*. The Dec phrase class covers value, function and type declarations which mostly bind values to identifiers. Both the alternates of *dec* that do not contain terminals are shown in the fragment in Figure 1 and we have also included the alternate that generates value declarations. The declarations in Dec invoke the expression phrase class, Exp, to define the syntax of values. For Exp there is a substantial difference between the full and bare languages. The former contains classical if and while constructs but the latter requires these to be translated into case statements. However, this does not impact on the grammar cycle analysis which is the same for both grammars. For any phrase class P there is also a phrase class Pseq that contains comma separated lists of elements of P. Figure 1 shows the rule for *tyvarseq*, the nonterminal corresponding to TyVarseq.

For the fullSMLphrase.bnf grammar *program* has two additional alternates

```

program ::= exp ; | exp ; program

```

that permit inputs to be expressions without requiring them to be coerced into declarations. The phrase class AtExp contains terminals whose patterns include identifiers and constants. For the full grammar there is a phrase class InfExp that

extends AtExp to include application expressions (AppExp) and infix operator expressions. The corresponding grammar nonterminals are *infexp*, *atexp*, and *appexp*.

The phrase class InfExp is the source of some lexical ambiguity in fullSMLchar.bnf and the additional alternates of *program* also increase the ambiguity of the grammar. But none of these extensions from bareSMLphrase.bnf to fullSMLphrase.bnf introduce any additional cyclic nonterminals or slots and our study applies to both grammars. The cyclic nonterminals are *topdec*, *strdec*, *spec* and *dec*.

The majority of the lexical tokens in fullSMLphrase.bnf are keywords and symbols that have a single associated lexeme. There are nine identifier and special constant tokens whose patterns contain multiple lexemes.

```

scon, lab, vid, tyvar, tycon,
funid, sigid, d, strid

```

The character level patterns for each of these tokens is specified using a lexical level grammar whose start nonterminal is the token. The lexical grammars are all ϵ -free, do not contain cycles, and have the property that the start symbols do not appear on the right hand side of any of the lexical level grammar rules. We call the union of these grammars lexicalChar.bnf.

We will discuss the results of applying cycle removal to SML derivations in Section 5. As part of our analysis we will use character level grammars, fullSMLchar.bnf and bareSMLchar.bnf, which are obtained from fullSMLphrase.bnf and bareSMLphrase.bnf by turning the nine tokens into nonterminals and adding the grammar rules in lexicalChar.bnf.

3 Shared packed parse forests (SPPFs)

In general it is not possible to determine whether a grammar is ambiguous. However, it is possible to tell whether a given input string, *u*, is ambiguous by using a general parser to construct all derivations of *u*, and disambiguation can be specified as rules for selecting one of these derivations.

A commonly used representation of the set of all derivation trees of *u* is a shared packed parse forest (SPPF) [3]. These are bipartite directed graphs are comprised of *non-packed nodes* that are the derivation tree nodes and *packed nodes* that group together families of children of the non-packed nodes. Disambiguation can be expressed as rules for selecting one packed node for each non-packed node. However, if the SPPF contains cycles, certain selections can result in a graph that does not correspond to derivations of *u*. The primary focus of this paper is an algorithm that removes all cycles from an SPPF and leaves a graph to which further node-removal disambiguation rules can be safely applied.

In this section we define binarised SPPFs and give some examples that illustrate issues that can arise from arbitrary cyclic node removal. In Section 4 we give the cycle removal algorithm and prove that it is ‘safe’ in the sense that the

resulting graph only embeds derivations of the input string and does embed at least one derivation.

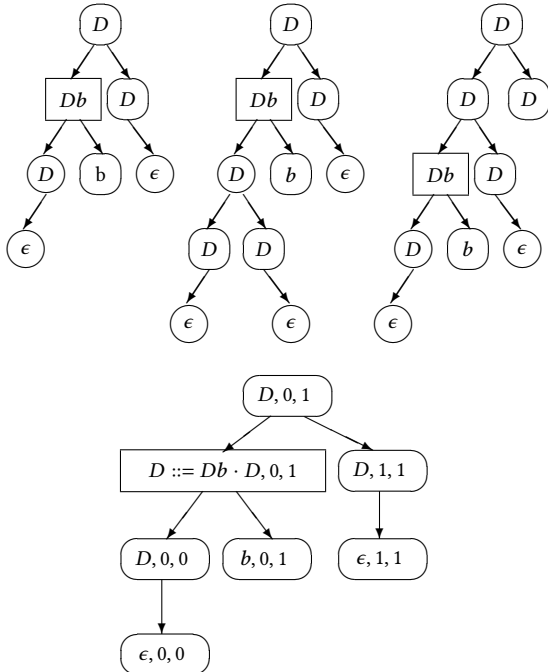
3.1 Binarised SPPFs

Derivations are naturally represented as rooted ordered trees in which interior nodes are labelled with nonterminals and leaf nodes are labelled with terminals or ϵ . The root node is labelled with the start nonterminal, the children of a node labelled X are labelled $x_1, \dots, x_k$, where $X ::= x_1 \dots x_k$ is a production rule, and, when read from left to right, the labels on the leaves form the input string.

In order to efficiently represent the potentially infinite set of all derivations of a sentence, derivation trees are binarised by introducing intermediate nodes so that packed nodes have at most two children. They are then combined into an SPPF so that common subtrees are shared. To construct an SPPF, integer extents are added to the labels of the derivation tree nodes. A node labelled (α, i, j) in an indexed derivation tree for $S \Rightarrow^* a_1 \dots a_n$ is the root of a subtree corresponding to a derivation $\alpha \Rightarrow^* a_{i+1} \dots a_j$. Intermediate nodes are labelled with grammar slots in the indexed trees. The following example is based on a simplified version of the fullSMLphrase.bnf rule for *dec*.

Example Γ_1 $D ::= a \mid \epsilon \mid DD \mid D b D$

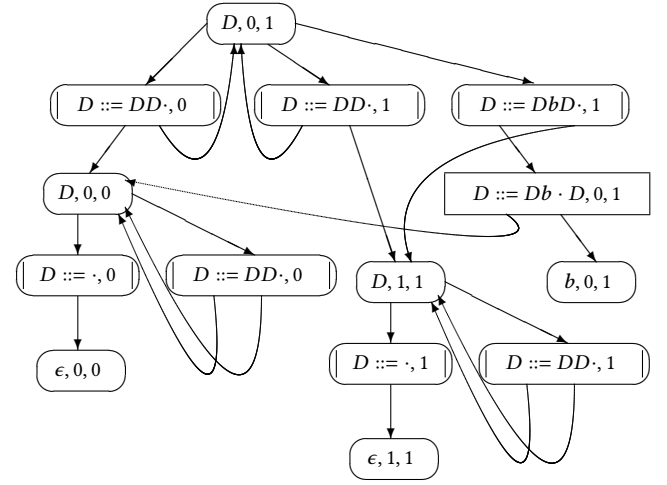
Below are three of the infinitely many binarised derivation trees for b in Γ_1 , followed by the indexed version of the left-most tree.



Indexed derivation trees are combined and for each derivation $A \Rightarrow^* a_{i+1} \dots a_j$ the associated set of children is grouped

under a packed node. For example, the graph below is the binarised SPPF embeds all the indexed versions of the binarised derivation trees, in Γ_1 , for b .

In detail, binarised SPPFs are ordered directed bipartite graphs of packed and non-packed nodes. The packed nodes have labels of the form $(X ::= \alpha \cdot \beta, k)$, where $X ::= \alpha\beta$ is a production rule. The non-packed nodes are symbol or intermediate nodes. Symbol nodes have labels of the form (x, i, j) where $x \in N \cup T \cup \{\epsilon\}$. Intermediate nodes have labels of the form $(X ::= \alpha \cdot \beta, i, j)$ where $\beta \neq \epsilon$ and $|\alpha| > 1$. We call i and j the left and right extents of (x, i, j) and k the pivot of $(X ::= \alpha \cdot \beta, k)$. In our diagrams the packed nodes have vertical lines at the edges of the ovals and the intermediate nodes are rectangular.



A packed node $w = (X ::= \alpha \cdot \beta, k)$ has either one or two children and exactly one parent. If $\beta = \epsilon$ then w 's parent is a symbol node (X, i, j) otherwise its parent is an intermediate node $(X ::= \alpha \cdot \beta, i, j)$. In either case $\alpha \Rightarrow^* a_{i+1} \dots a_j$. If $\alpha = \epsilon$ then $\beta = \epsilon$, $i = j = k$ and w has a single child (ϵ, k, k) . If $\alpha = x$, a terminal or nonterminal, then $\beta = \epsilon$, $i = k$ and w has a single child (x, k, j) . Otherwise $\alpha = \alpha_1 x$ and w has right child (x, k, j) . If $|\alpha_1| = 1$ then w has left child (α_1, i, k) , otherwise w has left child $(X ::= \alpha_1 \cdot x\beta, i, k)$.

It was noted by Johnson [4] that SPPFs are worst case unbounded polynomial in size, however binarising the initial indexed derivation trees results in SPPFs which are worst case cubic size [14]. SPPFs have the property that any derivation can be extracted by walking the SPPF, starting at the root node, and choosing a packed node child each time a non-packed node is encountered. It is a property of an SPPF that the input sentence is ambiguous if and only if there is at least one reachable SPPF node with more than one packed node child. Disambiguation can be specified in terms of packed node selection and the user can select the derivation which corresponds to their required semantics.

SPPF node derivation distance

In Section 2 we defined the derivation distance of one string

to another. We use this as the basis of the definition of the derivation distance of an SPPF node which is used in proofs, by contradiction, of aspects of the cycle removal algorithm's correctness.

The derivation distance of an SPPF symbol node $w = (x, i, j)$ is the derivation distance of $a_{i+1} \dots a_j$ from x , i.e., the smallest integer m such that $x \Rightarrow^m a_{i+1} \dots a_j$. The derivation distance is 0 if and only if w is a leaf node.

The derivation distance of an SPPF (intermediate) node $w = (X ::= \alpha \cdot \beta, i, j)$ is the derivation distance of $a_{i+1} \dots a_j$ from α . The derivation distance of w is 0 if and only if $\alpha \in \mathbf{T}^*$.

It is a property of SPPFs that the derivation distance exists for every non-packed node.

SPPF cycles

A cycle in an SPPF is a path from a node to itself, and as SPPFs are bipartite all cycles contain at least two nodes.

For an SPPF for $a_1 \dots a_n$, if a symbol node is labelled (X, i, j) then there is a derivation $X \Rightarrow^* a_{i+1} \dots a_j$. If X is cyclic then there is a derivation $X \Rightarrow^* \gamma^* X$ and hence a derivation $X \Rightarrow^* \gamma^* X \Rightarrow^* a_{i+1} \dots a_j$. The same SPPF node (X, i, j) will be used for both instances of X in this derivation and this node will therefore be in a cycle in the SPPF. Conversely, if (Y, p, q) is on a path from the node (X, i, j) we must have that there is a sub-derivation $X \Rightarrow^* \alpha Y \beta \Rightarrow^* \alpha a_{p+1} \dots a_q \beta \Rightarrow^* a_i \dots a_j$. Thus if (X, i, j) is in a cycle we have

$X \Rightarrow^* \alpha X \beta \Rightarrow^* \alpha a_{i+1} \dots a_j \beta \Rightarrow^* a_{i+1} \dots a_j$
so α and β are nullable and X is cyclic.

A similar argument shows that an intermediate node $(X ::= \alpha \cdot \beta, i, j)$ is in a cycle if and only if the slot $X ::= \alpha \cdot \beta$ is cyclic and there is a node (X, i, j) in the SPPF. Thus for an SPPF the cyclic non-packed nodes can be identified from the labels on the nodes. (Note this is not always true for subgraphs of the SPPF.) Also if the derivation distance of a node is 0 then it cannot be cyclic. So each non-packed cyclic node has a derivation distance m , where $m \geq 1$.

Derivations of an ambiguous string u can be discarded by removing corresponding SPPF packed nodes. For example, left-most derivations of b in Γ_1 of the form $D \Rightarrow DD \Rightarrow D \Rightarrow b$ can be discarded by removing the packed node labelled $(D ::= DD, 1)$ from the SPPF. In general, if $X ::= \alpha \mid \beta$ where both α and β can derive $a_{i+1} \dots a_j$, derivations

$X \Rightarrow \alpha \Rightarrow^* a_{i+1} \dots a_j$ may be discarded in favour of

$X \Rightarrow \beta \Rightarrow^* a_{i+1} \dots a_j$ by removing children of (X, i, j) labelled $(X ::= \alpha, k)$. However, for node removal to be 'safe' the resulting graph must retain the property that walking the graph, starting at the root node, choosing a packed node child of each non-packed node encountered and recursively walking its children, results in a derivation of u . For SPPFs that contain cycles removing nodes is not always safe in this sense, as we now discuss.

3.2 Cyclic SPPF examples

We need an SPPF cycle removal technique that works for all context free grammars even though most 'real' grammars do not display pathological properties. In this section we consider some examples using the SML-related Γ_1 from Section 3.1 and two somewhat contrived grammars that illustrate other difficult issues.

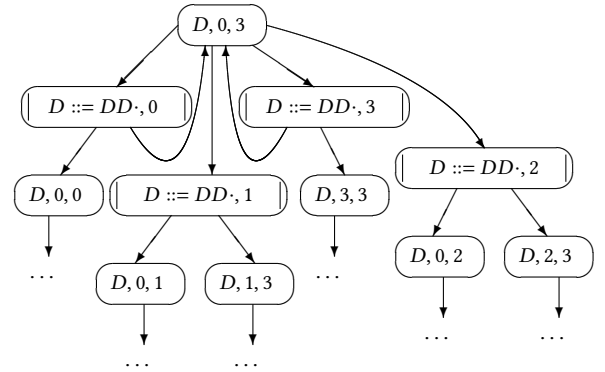
A common disambiguation strategy is, where there are two choices for a derivation step, to choose the one that matches the longest part of the input from the left. For an SPPF this concretises to choosing the packed node with the largest pivot k , because a packed node child $(X ::= \alpha x, k)$ of (X, i, j) corresponds to a derivation $X \Rightarrow^* \alpha x \Rightarrow^* a_{i+1} \dots a_j$ in which $\alpha \Rightarrow^* a_{i+1} \dots a_k$. The following example shows that longest match is not always safe.

Example Γ_1 with input aaa

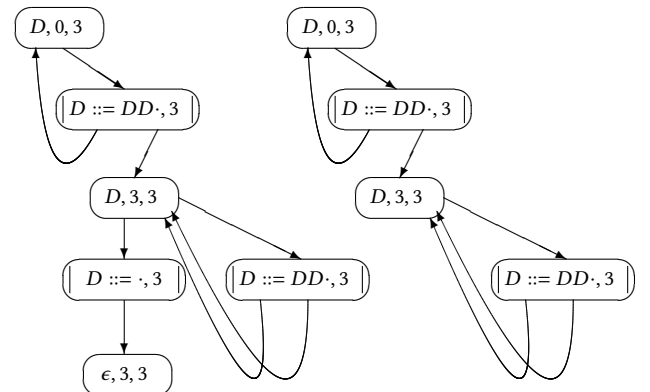
All derivations of aaa in Γ_1 , above, are of the form

$$D \Rightarrow DD \Rightarrow^* aaa.$$

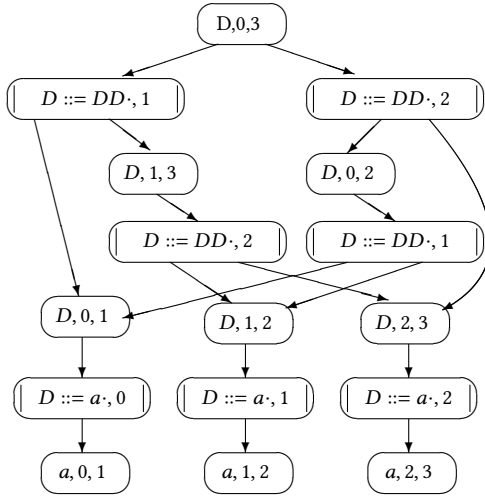
The left instance of D derives one of ϵ, a, aa or aaa , creating four corresponding packed nodes $(D ::= DD, 0)$, $(D ::= DD, 1)$, $(D ::= DD, 2)$, and $(D ::= DD, 3)$.



Longest match on these four nodes selects the graph on the left below and a further choice to select the node $(D ::= DD, 3)$ selects the graph on the right. Neither graph embeds any derivations.



However, if initially cycle removal is applied throughout the SPPF we get the graph

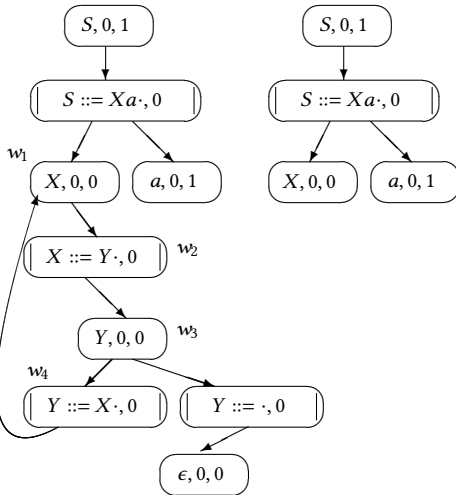


Longest match can be subsequently applied to select the packed node ($D ::= DD·, 2$) and the corresponding derivation tree for aaa .

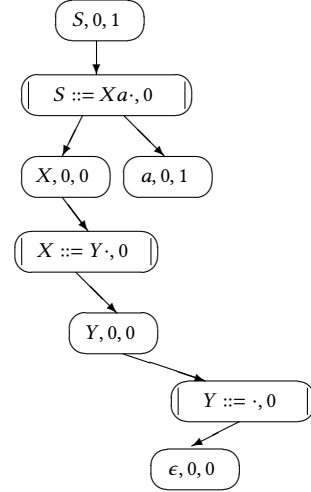
However, the next example shows that, in general, we cannot simply remove all the cyclic packed nodes from an SPPF.

Example Γ_2 $S ::= Xa \quad X ::= Y \mid b \quad Y ::= X \mid \epsilon$

The SPPF for a is on the left below, and deleting the cyclic packed node ($X ::= Y, 0$) results in the graph on the right, which is not a derivation tree.



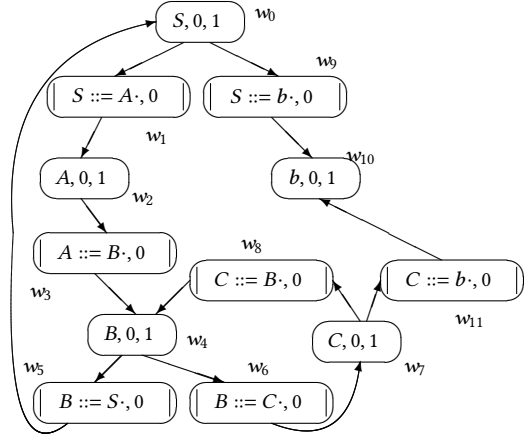
We can remove all cyclic packed nodes that have a non-cyclic sibling, and for the above example this would result in the graph



This gives an acceptable outcome for Γ_2 but it doesn't always remove all cycles. The next example shows it is possible for a cyclic packed node to have only cyclic siblings.

Example Γ_3 $S ::= A \mid b \quad A ::= B \mid a$
 $B ::= C \mid S \quad C ::= B \mid b$

The node w_4 in the SPPF for b has two packed node children, both of which are cyclic.



3.3 SubSPPFs of binarised SPPFs

Before giving an algorithm that does safely remove cycles from any SPPF we formalise what we mean by safe. Informally, by safe we mean that, if nodes are removed from an SPPF for u , the resulting graph still has the property that walking the graph, starting at the root node, and choosing a packed node child of each non-packed node encountered results in a derivation of u . We formalise this by defining the embedding of trees in a particular form of SPPF subgraph.

A subSPPF of an SPPF Δ is any non-empty graph Λ that can be obtained from Δ by removing zero or more packed nodes, together with their in and out edges and nodes which are then unreachable from the root, which has the property that every leaf node in Λ is also a leaf node in Δ .

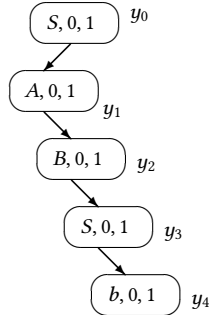
For a subSPPF Λ , let $NodeP(\Lambda)$ be the set of pairs (w, z) where w is a non-packed node in Λ and z is either a (packed node) child of w or, if w is a leaf node in Λ , a dummy node denoted by $\perp$.

We say that a subSPPF, Λ , embeds an ordered tree Ψ if there is a function f from nodes of Ψ to $NodeP(\Lambda)$ such that, for $f(y) = (w, z)$,

- if y is the root node of Ψ then w is the root node of Λ
- if z has left and right children, w_1 and w_2 , then y has left and right children y_1 and y_2 , and for some z_1, z_2 , $f(y_1) = (w_1, z_1)$ and $f(y_2) = (w_2, z_2)$
- if z has exactly one child, w_1 say, then y has exactly one child, y_1 , and for some z_1 , $f(y_1) = (w_1, z_1)$
- if $z = \perp$ (so w is a leaf node of Λ) then y is a leaf node of Ψ

Note, Ψ is a tree but the embedded image may not be a tree because $f(z)$ maybe cyclic. In general, it may be the case that $f(z) = f(y)$ but $z \neq y$.

Suppose that Λ is a subSPPF of the SPPF for u , and that f is an embedding of an ordered tree Ψ in Λ . We can obtain a labelled tree from Ψ by labelling each node y with the label of w , where $f(y) = (w, z)$. For example the labelled tree



is embedded in the SPPF for Γ_3 in Section 3.2 via the function $f(y_0) = (w_0, w_1)$ $f(y_1) = (w_2, w_3)$ $f(y_2) = (w_4, w_5)$ $f(y_3) = (w_0, w_9)$ $f(y_4) = (w_{10}, \perp)$

We say that Λ *respects derivations* (of u) if there is at least one labelled binarised derivation tree of u embedded in Λ and all the labelled binarised trees embedded in Λ are derivation trees of u .

A safe cycle removal algorithm is one which results in a subSPPF that respects derivations of u . The example for Γ_2 and input $u = a$ in Section 3.2 shows that it is possible for a subgraph obtained by removing cyclic nodes not to embed any trees at all. Specifying a safe cycle removal algorithm is therefore more subtle than initially may be expected.

4 Cycle breaking

We assume that we have a binarised SPPF Δ for u and that we want to construct a subSPPF Λ of Δ which is cycle free and respects derivations of u . The approach is, starting with $\Lambda = \Delta$, to consider, in turn, nodes in Λ which are cyclic in Δ , and either remove them from Λ or establish that they are no longer cyclic in Λ . Only packed nodes are removed by the

main part of the algorithm, unreachable nodes are removed from Λ as a final step. A node will only be established as no longer cyclic when all its remaining children have been established as being no longer cyclic. In fact, rather than removing nodes our algorithm marks all nodes in Δ as either for keeping or deletion and then constructs Λ by removing the nodes marked for deletion. A packed node will only be marked for deletion if it has at least one sibling marked for keeping, and once a node is marked the mark is never removed or overwritten.

Since all leaf nodes are non-cyclic we can consider the unmarked nodes in the SPPF from ‘the bottom up’, keeping a node if all its (current) children are marked for keeping. When there are no more nodes that can be kept, a packed node with a retained sibling is deleted. The underlying intuition comes from the observation, captured in Lemma 4.1, that in an SPPF there is a path from every node to a leaf node and thus every cycle has an ‘exit’ point, that is a non-packed node with a child that is not in the cycle. The proofs show that the algorithm finds these nodes.

Only cyclic packed nodes are considered for deletion, and they are retained if, when they are considered, they are already no longer in a cycle. This is to leave, where possible, disambiguation decisions for the language specifier to control in a post-cycle removal phase.

The details of exactly how SPPF nodes are marked for deletion will be dependent on the data structure implementation. Our algorithm maintains two sets, $\mathcal{X}$ and $\mathcal{D}$, of SPPF nodes, the former containing the cyclic nodes in the SPPF that are still ‘unmarked’ and the latter containing the packed nodes to be deleted. Nodes not in either $\mathcal{X}$ or $\mathcal{D}$ are effectively marked for keeping. All non-cyclic nodes are effectively marked for keeping before the algorithm starts.

We construct a graph, $\Delta_{\mathcal{Y}}$, from a set, $\mathcal{Y}$, of packed nodes in an SPPF Δ by removing the nodes $\mathcal{Y}$ (and all their in and out edges) and then removing any unreachable nodes.

In the following algorithm Δ is an SPPF, $children_{\Delta}(w)$ is the set of children of a node w in Δ and $sibling_{\Delta}(z)$ is the set of nodes that have the same parent, in Δ , as the packed node z . We let $\mathcal{P}_{\Delta}$ denote the set of packed nodes in Δ .

4.1 Cycle removal algorithm

1. Initialise $\mathcal{X}$ to be the cyclic nodes of Δ and initialise $\mathcal{D}$ to be empty
2. While $\exists(z \in \mathcal{X} \cap \mathcal{P}_{\Delta}) (sibling_{\Delta}(z) \not\subseteq \mathcal{X})$ {
 Pick a packed node $z \in \mathcal{X} \cap \mathcal{P}_{\Delta}$ with $sibling_{\Delta}(z) \not\subseteq \mathcal{X}$
 Add z to $\mathcal{D}$ and set $\mathcal{X} := \mathcal{X} \setminus \{z\}$ (i.e. remove z from $\mathcal{X}$)
 While $\exists(w \in \mathcal{X}) (children_{\Delta}(w) \cap \mathcal{X} = \emptyset)$ {
 Choose $w \in \mathcal{X}$ such that $children_{\Delta}(w) \cap \mathcal{X} = \emptyset$
 Set $\mathcal{X} := \mathcal{X} \setminus \{w\}$ (i.e. remove w from $\mathcal{X}$) } }
3. Output $\Delta_{\mathcal{D}}$

4.2 Algorithm safety

If an SPPF node is in a cycle so is one of its children and, conversely, if all the children of a node are no longer in cycles then nor is the parent. Since the SPPF is bipartite there are no ‘self loops’, no node is a child of itself, and so we can use ‘upwards propagation’, retaining a node if all its children are retained. It is easy to see that this approach works when every cyclic packed node has at least one non-cyclic sibling. The proof that it is safe even in cases where all the children of a node are cyclic is based on the property that, because of the nature of grammar derivations, any cycle will contain at least one non-packed node whose label generates a derivation that does not involve a cycle. The algorithm finds and retains such nodes, allowing their siblings to be removed.

Let Δ be an SPPF and let $u = a_1 \dots a_n$ be the corresponding input string. The following lemmas give a formal proof that the cycle removal algorithm above is safe in the sense that $\Delta_{\mathcal{D}}$ is a non-cyclic subSPPF that respects derivations of u .

Lemma 4.1. *Let $w = (\Omega, i, j)$ be a non-packed node in Δ with derivation distance $m \geq 1$. Then w has a child in Δ whose children all have derivation distance at most $m - 1$.*

Proof If $w = (X, i, j)$, where X is a nonterminal we have

$$X \Rightarrow x_1 \dots x_d \xRightarrow{m-1} a_{i+1} \dots a_j \quad (x_i \in \mathbf{N} \cup \mathbf{T})$$

and w has a child $z = (X ::= x_1 \dots x_d, k)$ corresponding to this derivation, which in turn has child $v = (x_d, l, j)$. If $d = 2$, z also has left child $v' = (x_1, i, k)$ and, if $d \geq 3$, z has left child $v' = (X ::= x_1 \dots x_{d-1} \cdot x_d, i, k)$. In either case the derivation distances of v and v' are at most $m - 1$.

If $w = (X ::= \alpha y x \cdot \beta, i, j)$, where $x, y \in \mathbf{N} \cup \mathbf{T}$, has derivation distance $m \geq 1$ we have

$$\alpha y x \Rightarrow x_1 \dots x_d \xRightarrow{m-1} a_{i+1} \dots a_j$$

and w has a child $z = (X ::= \alpha y x \cdot \beta, k)$ corresponding to this derivation. Then z has right child $v = (x, k, j)$ where $x \xRightarrow{*} a_{k+1} \dots a_j$ and v has derivation distance at most $m - 1$. If $\alpha = \epsilon$ then z has left child $v' = (y, i, k)$, otherwise z has left child $v' = (X ::= \alpha y \cdot x \beta, i, k)$. In either case v' has derivation distance at most $m - 1$.

Lemma 4.2. *The cycle break algorithm terminates with $\mathcal{X} = \emptyset$ and $\Delta_{\mathcal{D}}$ is a subSPPF of Δ which does not contain any cycles.*

Proof Both the inner and outer while loops remove an element from $\mathcal{X}$ at each iteration, so they both terminate.

Let $\mathcal{X}_1$ denote the set of elements in $\mathcal{X}$ when the algorithm terminates and suppose there is a node $w \in \mathcal{X}_1$, which must be cyclic in Δ . Since Δ is bipartite, if w is a packed node the (unique) parent of w is also cyclic and must also be in $\mathcal{X}$ because cyclic non-packed nodes are only removed from $\mathcal{X}$ if all their children have been removed. Thus we suppose w is a non-packed node, and that it is of minimum derivation

distance, m say, amongst the non-packed nodes in $\mathcal{X}_1$. Since w is cyclic $m \geq 1$. By Lemma 4.1, w has a packed node child of z whose children all have minimum distance less than m . By choice of w these children cannot be in $\mathcal{X}_1$, and thus z must have been removed from $\mathcal{X}$ by either the outer or inner while loop. This is a contradiction so $\mathcal{X} = \emptyset$ when the algorithm terminates.

By definition $\Delta_{\mathcal{D}}$ is obtained from Δ by deleting nodes, and the outer while loop ensures that all the nodes in $\mathcal{D}$ are packed nodes. We note that only elements currently in $\mathcal{X}$ are added to $\mathcal{D}$ so if at any step in the algorithm an element is not in $\mathcal{D} \cup \mathcal{X}$ then it will never be added to $\mathcal{D}$. Then we can see all nodes in $\mathcal{D}$ have at least one sibling not in $\mathcal{D} \cup \mathcal{X}$ by induction on the outer while loop steps. This is true for the first iteration as $\mathcal{D} = \emptyset$. When w is added to $\mathcal{D}$ it must have a sibling, z , which is not in $\mathcal{X}$. If z is in $\mathcal{D}$ then it was added at a previous step so by induction z has a sibling y which is not in $\mathcal{D} \cup \mathcal{X}$, and $y \neq w$. As y is a sibling of w this shows that w has at least one sibling not in $\mathcal{D}$. So no non-leaf nodes in Δ become leaf nodes in $\Delta_{\mathcal{D}}$, and $\Delta_{\mathcal{D}}$ is a subSPPF of Δ .

Any cycle in $\Delta_{\mathcal{D}}$ consists of nodes that are cyclic in Δ , and hence belong to $\mathcal{X}$ at the start of the algorithm, but are not in $\mathcal{D}$. So any node w which is cyclic in $\Delta_{\mathcal{D}}$ was removed from $\mathcal{X}$ by the inner while loop of the cycle break algorithm.

Let us suppose that of all the cyclic nodes in $\Delta_{\mathcal{D}}$, w was the first node to be removed from $\mathcal{X}$. Since the graph is bipartite there are no self loops, and w must have a child z in $\Delta_{\mathcal{D}}$ which is also cyclic. From the condition for removal from $\mathcal{X}$, z must have already been removed from $\mathcal{X}$ and by the inner while loop as z is in $\Delta_{\mathcal{D}}$. This is contrary to the choice of w , so $\Delta_{\mathcal{D}}$ is a subSPPF and does not contain any cycles.

Lemma 4.3. *$\Delta_{\mathcal{D}}$ respects derivations of $a_1 \dots a_n$.*

Proof Suppose that f is an embedding of an ordered tree Ψ in $\Delta_{\mathcal{D}}$ as defined in Section 3.3, and that a node y of Ψ is labelled with the label of w , where $f(y) = (w, z)$.

Since Ψ is a tree each node y in Ψ has a well defined *depth*, d , which is the length of the longest path from y to a leaf node of Ψ . We show by induction on d that if $y = (X, i, j)$ then the tree rooted at y is a binarised derivation tree for $X \xRightarrow{*} a_{i+1} \dots a_j$, and if $y = (X ::= \alpha \cdot \beta, i, j)$ then the tree rooted at y is a binarised derivation tree for $\alpha \xRightarrow{*} a_{i+1} \dots a_j$.

If $d = 0$ then y is a leaf node of Ψ and hence of $\Delta_{\mathcal{D}}$, and thus, since $\Delta_{\mathcal{D}}$ is a subSPPF, of Δ . So either $y = (a_i, i, i + 1)$ or $y = (\epsilon, i, i)$ and in either case y is the root of a (trivial) derivation tree for $a_{i+1} \dots a_j$.

Now suppose that the result is true for nodes of depth less than d and that y has depth $d \geq 1$.

If $y = (X, i, j)$, and $f(y) = (w, z)$, then $w = (X, i, j)$ and because Δ is an SPPF for u , $z = (X ::= \alpha \cdot, k)$ and $X \xRightarrow{*} \alpha \xRightarrow{*} a_{i+1} \dots a_j$. Either z has exactly one child $z_1 = (\alpha, i, j)$ or left child $z_1 = (\alpha', i, k)$ and right child $z_2 = (x, k, j)$, where

$\alpha = \alpha'x$. Then by the definition of an embedding, correspondingly y has exactly one child, $y_1 = (\alpha, i, j)$ or left child $y_1 = (\alpha', i, k)$ and right child $y_2 = (x, k, j)$. Since Ψ is a tree the depths of the children of y are less than d , so by induction they are the roots of binarised derivation trees for $\alpha \Rightarrow^* a_{i+1} \dots a_j$ in the case of one child, and $x \Rightarrow^* a_{k+1} \dots a_j$ and $\alpha' \Rightarrow^* a_{i+1} \dots a_k$ in the case of two children. Thus y is the root of a binarised derivation tree for $y \Rightarrow^* a_{i+1} \dots a_j$.

If y is the root of Ψ then w is the root of $\Delta_{\mathcal{D}}$. Since $\Delta_{\mathcal{D}}$ is a subSPPF, $w = (S, 0, n)$. Thus Ψ is a derivation tree for $S \Rightarrow^* u$, as required.

To show that $\Delta_{\mathcal{D}}$ embeds a derivation tree for u we note the following properties. Let $\Delta'_{\mathcal{D}}$ denote the subtree obtained just by removing the nodes in $\mathcal{D}$ with their in and out edges. The cycle removal algorithm ensures that every non-packed node in $\Delta'_{\mathcal{D}}$ which has a child in Δ also has at least one child in $\Delta'_{\mathcal{D}}$. Since $\mathcal{D}$ only contains packed nodes, the children of a packed node in $\Delta'_{\mathcal{D}}$ are the same as its children in Δ . Since children of reachable nodes are reachable, these properties remain true for $\Delta_{\mathcal{D}}$ which is obtained from $\Delta'_{\mathcal{D}}$ by removing unreachable nodes.

Construct a tree Ψ and an embedding f as follows. Create a node y_0 in Ψ and define $f(y_0) = (w_0, z_0)$, where w_0 is the root node of $\Delta_{\mathcal{D}}$ and z_0 is a child of w_0 . For each child, w , of z_0 create a new node y in Ψ as a child of y_0 , and define $f(y) = (w, z)$, where z is a child of w if it has one and $z = \perp$ otherwise. Carry on in this way, adding new nodes as children for nodes created in Ψ until the only leaf nodes are nodes y such that $f(y) = (w, \perp)$. Since $\Delta_{\mathcal{D}}$ does not contain any cycles, the length of paths in $\Delta_{\mathcal{D}}$ is bounded (by the depth of the root), so the construction process results in a finite graph Ψ , which is a rooted ordered tree. By construction Ψ is embedded in $\Delta_{\mathcal{D}}$ and it is also a derivation tree for u , by the argument above.

4.3 Examples

Example Γ_2 For the Example Γ_2 in Section 3.2, we have that the algorithm starts with $\mathcal{X} = \{w_1, w_2, w_3, w_4\}$. The only one of these that has a sibling not in $\mathcal{X}$ is w_4 , so this is removed from $\mathcal{X}$ and added to $\mathcal{D}$,

$$\mathcal{X} = \{w_1, w_2, w_3\} \quad \mathcal{D} = \{w_4\}$$

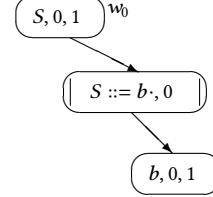
For the inner loop we now have that none of the children of w_3 is in $\mathcal{X}$, so it will be removed from $\mathcal{X}$. On the next iteration of the inner loop w_2 is removed from $\mathcal{X}$, and finally w_1 is removed, giving the sequence

$$\mathcal{X} = \{w_1, w_2\} \quad \mathcal{X} = \{w_1\} \quad \mathcal{X} = \{\}$$

The algorithm terminates with $\mathcal{D} = \{w_4\}$. This node is removed, and all other nodes are still reachable so the subSPPF constructed is the one shown in Example Γ_2 in Section 3.2.

Example Γ_3 We have implemented the cycle removal algorithm in the ART tool [5]. For Example Γ_3 in Section 3.2 the

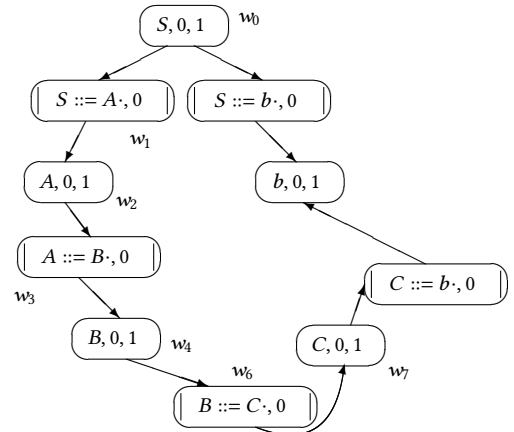
algorithm starts with $\mathcal{X} = \{w_0, w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8\}$. The ART implementation adds w_1 to $\mathcal{D}$ on the first iteration of the outer loop, and then the inner loop removes w_0 followed by w_6 . The second iteration of the outer loop adds w_8 to $\mathcal{D}$ and then the inner loop removes the remaining elements from $\mathcal{X}$. This results in the subSPPF



But the first iteration of the outer while loop could have selected w_8 . The inner loop would then remove w_7 and w_6 .

$$\mathcal{D} = \{w_8\} \quad \mathcal{X} = \{w_0, w_1, w_2, w_3, w_4, w_5\}$$

The second iteration of the outer while loop could then have selected w_5 , and the inner loop would then have removed all the remaining elements from $\mathcal{X}$. So $\mathcal{D} = \{w_8, w_5\}$, giving the subSPPF



In general then, the subSPPF constructed can depend on the order in which the packed nodes in $\mathcal{X}$ are considered. There is no ‘correct’ choice, cycle breaking will always remove some derivations, and there are grammars for which to break a cycle a non-cyclic derivation must be removed.

5 Cycle removal in SML

The cycles that can arise in SPPFs for SML are more restricted than those that can arise in the general case. In this section we will discuss the properties of SML SPPF cycles.

5.1 Cycles in SML SPPFs

Any SPPF analysis is in terms of a particular grammar. As we discussed in Section 2.1 the grammar fullSMLphrase.bnf we have used for our study is the same as the 1997 SML Definition [9] except that we have ‘multiplied out’ optional symbol instances and defined sequences using recursive rules to obtain pure BNF. We also considered what is essentially a subgrammar, bareSMLphrase.bnf, which defines the subset of

the full SML language on which semantics are specified. The analysis in this section applies to both grammars, and to the corresponding character level grammars. All the grammars have the same cyclic nonterminals and slots.

A nonterminal X is cyclic if and only if there is a derivation

$$X \Rightarrow \alpha Y \beta \stackrel{*}{\Rightarrow} X$$

in which $\alpha \stackrel{*}{\Rightarrow} \epsilon$, $\beta \stackrel{*}{\Rightarrow} \epsilon$ and $Y \stackrel{*}{\Rightarrow} X$.

We can define a relation on the grammar non-terminals under which X is related to Y if there is a grammar rule $X ::= \alpha Y \beta$ where α, β are nullable. It is therefore straightforward to compute the cyclic nonterminals and slots in a grammar. If $X \stackrel{+}{\Rightarrow} Y$ and $Y \stackrel{+}{\Rightarrow} X$ we say that X is cyclically related to Y .

Using ART we can confirm that the nonterminals

topdec strdec dec spec

are the only cyclic nonterminals in `fullSMLphrase.bnf` and, apart from the `Pseq` class of variables that define sequences of elements from a phrase class `P`, these are the only nullable nonterminals. We can also get ART to identify the sets $ntd(X)$ of nonterminals Z such that $X \stackrel{+}{\Rightarrow} Z$, and for the cyclic nonterminals we have

$$ntd(topdec) = \{topdec, strdec, sigdec, fundec, dec\}$$

$$ntd(strdec) = \{strdec, dec\}$$

$$ntd(dec) = \{dec\}$$

$$ntd(spec) = \{spec\}$$

We can see that each of the cyclic nonterminals is only cyclically related to itself.

The ART analysis also confirms that cyclic slots are

$$topdec ::= strdec topdec \cdot$$

$$strdec ::= strdec strdec \cdot$$

$$dec ::= dec dec \cdot$$

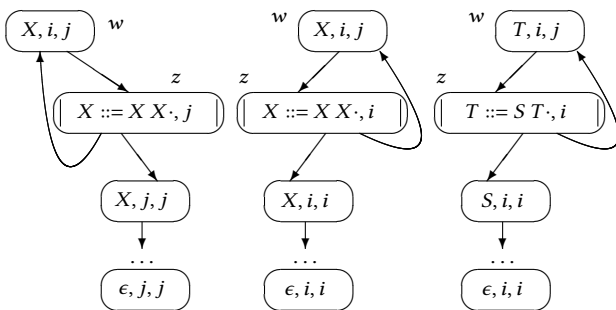
$$spec ::= spec spec \cdot$$

$$strdec ::= strdec \cdot strdec$$

$$dec ::= dec \cdot dec$$

$$spec ::= spec \cdot spec$$

the top four of which are the ones that can label SPPF nodes. Thus all the cycles in any SPPF for the grammar have one of the forms



where $T = topdec$, $S = strdec$ and $X \in \{strdec, dec, spec\}$.

The cycle removal algorithm removes all the elements from the initial set $\mathcal{X}$ of cyclic nodes. A node is only removed from the set $\mathcal{X}$ at the inner loop if all its children have already

been removed. So the nodes w can only be removed when their child z has been removed. Since w is a child of z , the node z can only be removed by the outer loop, and so z will be added to $\mathcal{D}$.

Thus the cyclic removal algorithm will always put all the cyclic packed nodes of an SML SPPF into $\mathcal{D}$. Then the output subSPPF $\Delta_{\mathcal{D}}$ is independent of the order in which nodes are processed, and $\Delta_{\mathcal{D}}$ has had all the cyclic packed nodes from the original SPPF removed.

We also note that, as a cyclic packed node will only be removed from $\mathcal{X}$ by the outer loop of the cyclic node removal algorithm, it must already have a sibling not in $\mathcal{X}$. Since all cyclic packed nodes are added to $\mathcal{D}$ every cyclic packed node must have a sibling which is not in the initial $\mathcal{X}$ and is therefore not cyclic and is not in $\mathcal{D}$.

5.2 Character level grammars

The definition of node removal safety for an SPPF Δ for u is that the resulting graph Λ respects derivations of u . That is, Λ embeds only derivation trees of u and it embeds at least one derivation tree. SPPFs are correct in the sense that if a string u is in the language generated by a grammar Γ then the SPPF for u with respect to Γ respects derivations and embeds all the binarised derivation trees of u . The safety property of the cycle removal algorithm ensures correctness in the sense that the language represented by the yields of the cycle-removed graphs is the same as $L(\Gamma)$. However, some of the derivations of a string u will be ‘lost’ and if semantics specification is based on the syntax grammar Γ , as it is for the SML Definition, some potential semantic interpretations may be lost.

Of course, if a general parsing technique is being used disambiguation is not required for the correctness of the parser. The purpose of disambiguation is precisely to reduce the number of semantic interpretations that have to be considered by a semantics analyser. Thus ambiguity reduction via SPPF node removal is considered safe even if derivations are lost, as long as a derivation is retained and no new sentences are created. It is for the specification designer to consider how their specified node removal will impact on the consequent semantics specification. But for character level grammars lexicalisation is effectively part of the syntax specification. Thus lexicalisation choices may be impacted by syntax level disambiguation and hence by cycle removal.

In detail, it is usual, as described for example in [2], for language syntax specifications to consist of the specification of a set ‘words’, lexical tokens, and a set of ‘sentences’ over the tokens, the phrase level specification. We call the set of those character strings that belong to a token its *pattern*. The lexicalisation of input character strings, that is the translation of character strings into token strings, is not normally unique and the lexer typically makes some disambiguation decisions.

This approach is problematic for specifications, such as the SML Definition, in which there is significant overlap between

the patterns of different tokens. For example, in SML **vid**, **strid**, **funid**, **sigid** and **tycon** all contain the set of C-style identifiers that begin with a letter. In most cases replacing one of these tokens with another results in a token string that is not syntactically correct, so the parser can resolve the lexical ambiguity. An approach that can be used in such a situation is to use a so-called character level grammar.

In a character level specification there is no initial lexical specification. The tokens are treated as grammar nonterminals, which we call paraterminals, and grammar rules define their patterns. As we mentioned, we have taken this approach to create the grammars `fullSMLchar.bnf` and `bareSMLchar.bnf`. The lexicalisations are the sentential forms over the paraterminals, that is the paraterminal strings α such that $S \xRightarrow{*} \alpha$. Thus, for character level grammars, lexicalisation becomes part of the semantics and the removal of derivations can remove sentential forms and therefore certain lexicalisations.

For the SML Definition there is a small class of lexical ambiguities that are not resolved by the phrase level specification and need to be resolved as part of the post-parse semantic disambiguation phase. These mainly arise because `=` can be both a keyword which is used as part of a declaration, as in the grammar rule for `valbind`, and also an identifier, in the lexical class `Vid`, which denotes the equality predicate. This results, for example, in two syntactically valid lexicalisations of the declaration `val a=b=c`;

```
val vid = vid vid vid ;
val vid vid vid = vid ;
```

The safety of the cycle removal algorithm is not sufficient to ensure, in general, that no syntactically valid lexicalisations are lost from a character level grammar. However, the nature of the cycles in `fullSMLchar.bnf` is limited. As we have remarked, every SPPF node with packed node children has at least one child that is not in $\mathcal{D}$. Since the non-packed node descendants of a node (Ω, i, i) are all of the form (Ω', i, i) , the only non-packed nodes that are deleted, because of the reachability step, are roots of derivation trees for ϵ . In particular, any SPPF node (Ω, i, j) where $i \neq j$ remains in the cycle removed subSPPF, and it retains all its non-cyclic children. The patterns of tokens do not contain ϵ , and so all the nodes in the SPPF that are labelled with paraterminals are also nodes in the cycle removed subSPPF.

If $t_1 \dots t_d$ is a syntactically valid lexicalisation of a character string u then there is a derivation

$$program \xRightarrow{*} t_1 \dots t_d \xRightarrow{*} u$$

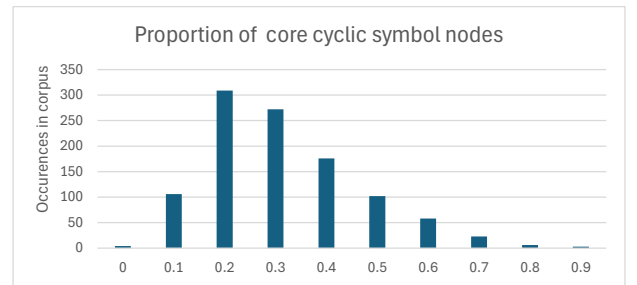
in the grammar `fullSMLchar.bnf`, and we can choose this derivation to be cycle-free. The corresponding derivation tree, Ψ , is embedded in the SPPF. In general, as we saw for Γ_3 in Section 4.3, removing cyclic packed nodes can remove non-cyclic derivations. But our analysis shows that, because of the restricted nature of cyclic packed nodes in the SML

SPPFs, no cyclic packed node is involved in the tree for a non-cyclic derivation and so no node in Ψ is included in the delete set $\mathcal{D}$. By induction on the depth of the nodes we can see that none of the nodes in Ψ are removed by the reachability step. So the lexicalisation $t_1 \dots t_d$, and all lexicalisations, are retained in $\Delta_{\mathcal{D}}$.

5.3 Prototype implementation and the MLWorks source code

Our focus in this paper is primarily theoretical, however we have made a prototype implementation of the SPPF cycle removal algorithm described above so as to investigate the behaviour of cycle removal on real SML program source. In this section we use the `fullSMLchar.bnf` described above, which has 77 nonterminals, and we give observations based on an MLWORKS corpus of 1,081 code files published at <https://github.com/Ravenbrook/mlworks>. Each file has been white-space compressed by replacing any run of two or more whitespaces with a single whitespace, unless the run contains a newline character in which case the run is replaced with a single newline character. Fourteen of the files in the corpus fail to parse due to the presence of SML extension features not defined in the standard grammar, leaving an experimental corpus of 1,067 files. The full set of files used here is available within our *Reference Language Corpora* site <https://github.com/AJohnstone2007/referenceLanguageCorpora/tree/main/languages/sml/corpus/MLWorks/sle25CycleBreak>

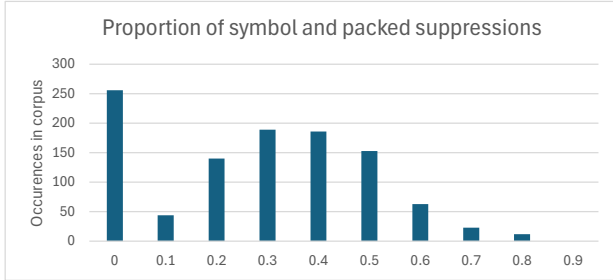
We begin by measuring the proportion of cyclic nodes in SML SPPFs. The *core* of an SPPF is that set of nodes which are reachable from the root node labelled $(S, 0, n)$ where S is the start symbol and n is the length of the input — all derivations embedded in the SPPF will have this node as their root, and thus non-reachable nodes cannot be part of any derivation, so we exclude them. The frequency distribution shows the proportion of core non-packed nodes which are cyclic in our experimental corpus.



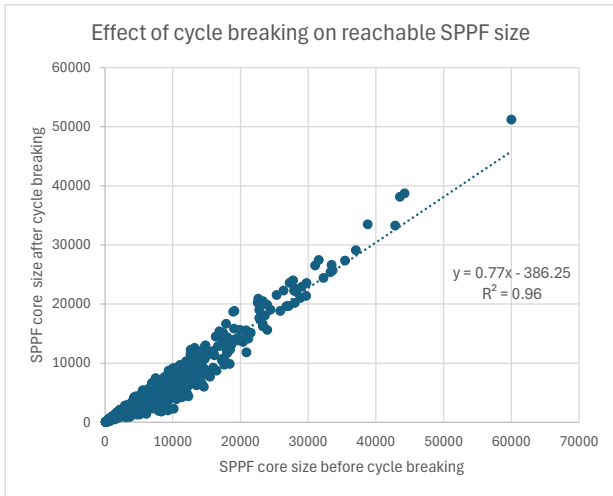
The mean of this distribution is 0.266, but as is clear from the figure, there are large numbers of example files for which the proportion is more than twice that.

As we have described, the cycles in the SML grammar rules are quite limited, but they affect declarations and of course declarations are ubiquitous in programs, hence the high observed cyclic:non-cyclic ratios.

This high proportion of cyclic nodes then drives a substantial reduction in the size of the SPPF as a result of cycle removal. This frequency analysis shows the proportion of all core nodes that are suppressed by the algorithm; that is we include packed nodes here as well so as to capture the effect on ambiguity reduction.



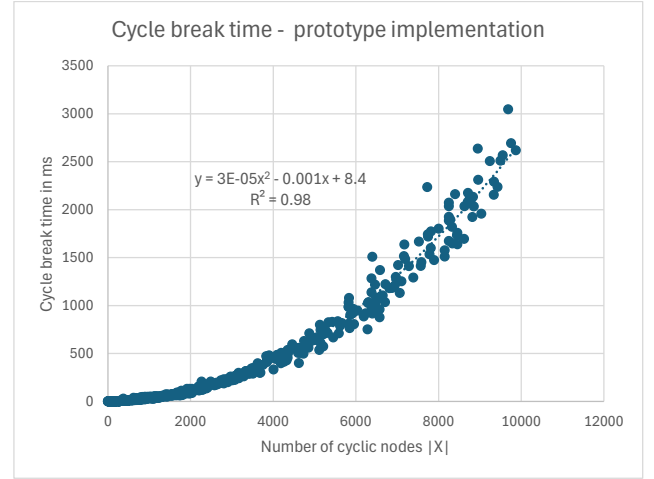
Here is the same data, presented as graph of core SPPF size before and after cycle breaking. The trend line shows that on average roughly 25% of nodes are suppressed but the data points for long files are mostly above trend, indicating that for longer files the suppressed proportion reduces. We speculate that this is because there are many short files made up almost entirely of declarations.



In this theory-focussed paper we have not attempted a high-performance implementation, and indeed the run times for some examples would be unacceptable in a production environment though adequate for this investigation. An obvious improvement would be to use more efficient data structures than the standard Java API methods, which we shall do for future investigations of the algorithm.

The performance of the present algorithm is dominated by the need for a (truncated) search of X for each element of X , generating a worst-case $O(|X|^2)$ complexity, which is reflected in this display of run time vs. $|X|$, $|X| < 10,000$. In future work we shall analyse the impact of this algorithm on (a) the number of sentences embedded in the SPPF and

(b) the interaction between this algorithm and our existing disambiguation mechanisms.



6 Concluding remarks

We have presented an algorithm that safely removes cyclic nodes from an SPPF for u without removing all the derivations of u or introducing derivations of other strings. The algorithm results in a subSPPF to which further disambiguation choices such as longest match can then be applied.

This is a theory paper and we have not attempted to make the cycle removal algorithm efficient, although we have shown that the runtimes for SML are manageable. If required it would be possible to make the algorithm more efficient, for example by exploiting the fact that when a node is removed from X only its parent nodes can become new candidates for removal from X .

Our algorithm structure, in which nodes are deleted only on the outer loop, favours retaining derivations where there is a choice. Only cyclic packed nodes are considered for deletion, and they are retained if, when they are considered, they are already no longer in a cycle. However, it is easy to modify the algorithm so that all cyclic packed nodes are deleted except where this would remove all the children of a node. This may be more appropriate for a less experienced or student user who wants something that ‘just works’, removing as much ambiguity as possible.

Although the algorithm favours derivation retention, as shown for the example Γ_3 in Section 4.3, the particular derivations retained can depend on the order in which the elements of X are considered. It is possible to consider all sets $\mathcal{D}$ such that $\Delta_{\mathcal{D}}$ is cycle-free, and this would give insight to potential heuristics that could direct the order in which nodes in X are considered. However, it should be noted that there exist grammars for which, to break the cycles, a non-cyclic derivation must be removed so no heuristic can be ‘perfect’. For SML, which is the focus of this paper, all cyclic packed nodes must be removed anyway, so choices do not arise.

References

- [1] 2013. *HaMLet; SML reference interpreter*. hpeople.mpi-sws.org/~rossberg/hamlet/.
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: principles, techniques, and tools, 2nd edition*. Addison-Wesley.
- [3] Sylvie Billot and Bernard Lang. 1989. The Structure of Shared Forests in Ambiguous Parsing. In *Proceedings of the 27th conference on Association for Computational Linguistics*. Association for Computational Linguistics, 143–151.
- [4] Mark Johnson. 1991. The computational complexity of GLR parsing. In *Generalized LR parsing*, Masaru Tomita (Ed.). Kluwer Academic Publishers, The Netherlands, 35–42.
- [5] Adrian Johnstone and Elizabeth Scott. 2011. Translator generation using ART. In *SLE 2010 (Lecture Notes in Computer Science, Vol. 6563)*, M.van den Brand B.Malloy, S.Staab (Ed.). Springer-Verlag, 306–315.
- [6] Stefan Kahrs. 1994. Mistakes and Ambiguities in the Definition of Standard ML. www.cs.kent.ac.uk/pubs/1993/569/, University of Edinburgh Technical Report ECS-LFCS-93-257.
- [7] Lennart C. L. Kats and Eelco Visser. 2010. The Spoofax language workbench: rules for declarative specification of languages and IDEs. In *Proceedings of the 25th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2010*. ACM Press, 444–463. doi:doi.org/10.1145/1869459.1869497
- [8] P. Klint, T. van der Storm, and J. Vinju. 2009. Rascal: A domain specific language for source code analysis and manipulation. In *Source Code Analysis and Manipulation*. IEEE, 108–177.
- [9] Robin Milner, Mads Tofte, Robert Harper, and David MacQueen. 1997. *The Definition of Standard ML (Revised)*. MIT Press, Cambridge, MA.
- [10] Andreas Rossberg. 2013. Defects in the Revised Definition of Standard ML. people.mpi-sws.org/~rossberg/papers/sml-defects-2013-09-18.pdf.
- [11] Elizabeth Scott and Adrian Johnstone. 2013. GLL parse-tree generation. *Science of Computer Programming* 78 (2013), 1828–1844. doi:[10.1016/j.scico.2012.03.005](https://doi.org/10.1016/j.scico.2012.03.005)
- [12] Elizabeth Scott and Adrian Johnstone. 2018. GLL syntax analysers for EBNF grammars. *Science of Computer Programming* 166 (2018), 120–145. doi:[10.1016/j.scico.2018.06.001](https://doi.org/10.1016/j.scico.2018.06.001)
- [13] Elizabeth Scott and Adrian Johnstone. 2023. Analysing the SML97 Definition: Lexicalisation. In *Eelco Visser Commemorative Symposium (EVCS 2023) (Open Access Series in Informatics (OASIs), Vol. 109)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 23:1–23:12.
- [14] Elizabeth Scott, Adrian Johnstone, and Giorgios Economopoulos. 2007. A cubic Tomita style GLR parsing algorithm. *Acta Informatica* 44, 6 (2007), 427–461. doi:[10.1007/s00236-007-0054-z](https://doi.org/10.1007/s00236-007-0054-z)
- [15] Elizabeth Scott, Adrian Johnstone, and Robert Michael Walsh. 2023. Multiple input parsing and lexical analysis. *ACM Transactions on Programming Languages and Systems* 45 (2023), 14:1–14:44. doi:doi.org/10.1145/3594734
- [16] Masaru Tomita. 1991. *Generalized LR parsing*. Kluwer Academic Publishers, The Netherlands.
- [17] M.G.J. van den Brand, J. Heering, P. Klint, and P.A. Olivier. 2002. Compiling language definitions: the ASF+SDF compiler. *ACM Transactions on Programming Languages and Systems* 24, 4 (2002), 334–368.

Received 2025-03-07; accepted 2025-04-01